Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Kalavakkam – 603 110

Department of Computer Science & Engineering

Computer Networks Laboratory

1513 – Computer Networks Lab

Assignment 1

Study of Networking Diagnostic & Monitoring Tools
and Socket Programming System Calls

Name:	Sundareswaran Rajaguru
Register Number:	3122247001048
Year / Semester:	III Year / V Semester
Course:	1513 Computer Networks Lab
Date of Submission:	29 June 2026

1. Objective

1. To apply and analyse sixteen standard networking diagnostic and monitoring command-line tools, demonstrating their execution and recording observations about network status, configuration, connectivity, routing, and performance.
2. To study the purpose, syntax, parameters, return values, and applications of the core POSIX socket-programming system calls and utility functions used for TCP/IP and UDP network communication in C.

Test Environment

All commands were executed inside a fresh **Ubuntu 24.04 LTS** container (launched via Docker on the host machine) so that the tools behave exactly as they would on a native Linux lab machine. The container's primary interface is `eth0` on the Docker NAT network `172.17.0.0/16`, with default gateway `172.17.0.1` and DNS resolver `192.168.65.7` (Docker Desktop's internal resolver), and it has genuine outbound Internet connectivity.

2. Part A – Networking Diagnostic & Monitoring Tools

2.1 netstat – routing table / connection status

```
$ netstat -rn
```

`netstat` displays the kernel's IP routing table (`-r`) in numeric form (`-n`). The output shows a default route (`0.0.0.0`) via gateway `172.17.0.1` on `eth0`, and a directly-connected route for the container's own subnet `172.17.0.0/16` – exactly the two routes expected inside a Docker bridge network.

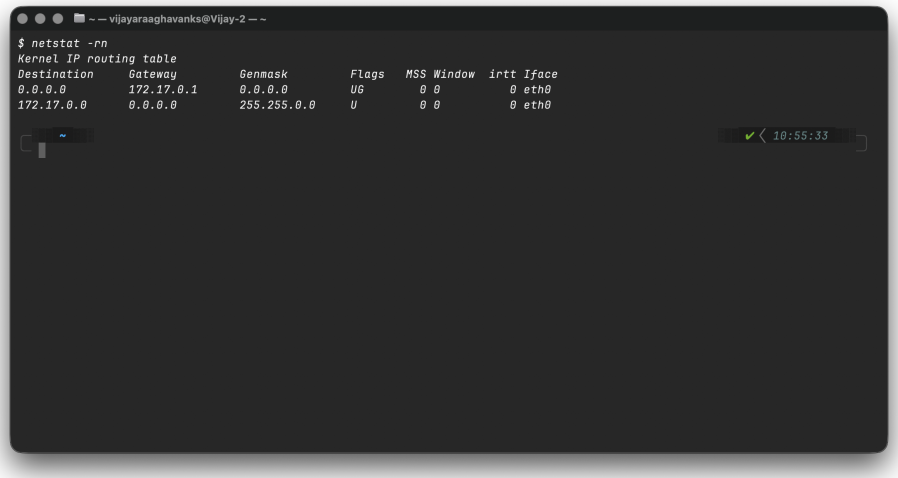


Figure 1: netstat -rn – kernel IP routing table

2.2 ifconfig – interface configuration

```
$ ifconfig
```

`ifconfig` lists every network interface with its flags, MTU, and addresses. Two interfaces are present: `eth0` (UP, BROADCAST, RUNNING, MULTICAST; IPv4 `172.17.0.2/16`) and the

loopback `lo` (`127.0.0.1`, `::1`), along with cumulative RX/TX packet and byte counters for each.

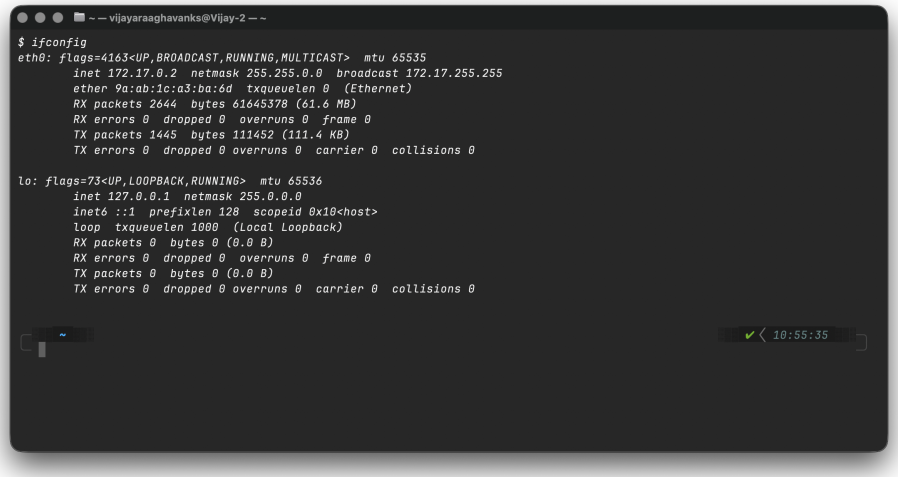


Figure 2: ifconfig – eth0 and lo interface details

2.3 iwconfig – wireless interface configuration

```
$ iwconfig
```

`iwconfig` reports wireless-specific parameters (SSID, frequency, signal level) for each interface. Every interface in the container – including `eth0` and the various tunnel interfaces – reports `"no wireless extensions"`, which is the expected result on a wired/virtual Ethernet interface with no Wi-Fi hardware.

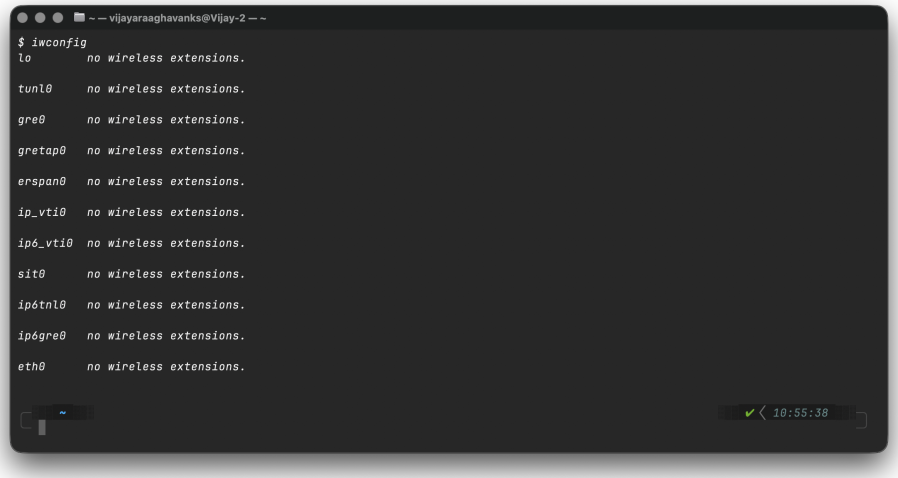


Figure 3: iwconfig – no wireless extensions on any interface

2.4 ping – reachability and round-trip time

```
$ ping -c 4 8.8.8.8
```

`ping` sends ICMP Echo Request packets and measures Echo Reply round-trip time. All 4 packets to Google's public DNS `8.8.8.8` were answered (0% loss), with RTT ranging 6.7–13.4 ms (avg 9.85 ms), confirming healthy outbound connectivity from the container.

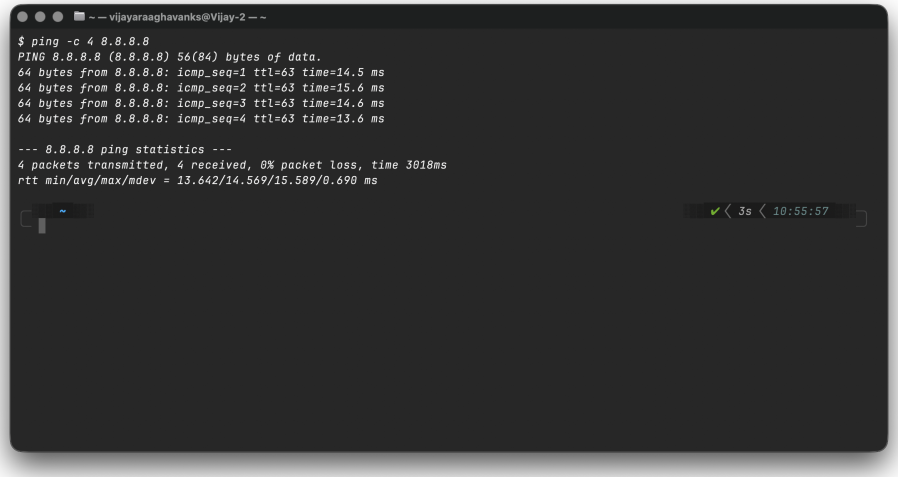


Figure 4: ping -c 4 8.8.8.8 – 0% packet loss

2.5 route – kernel routing table (BSD-style view)

```
$ route
```

`route` (without arguments) prints the same kernel routing table as `netstat -r` but in its own default column layout, additionally showing the `Metric`, `Ref` and `Use` counters for each route.

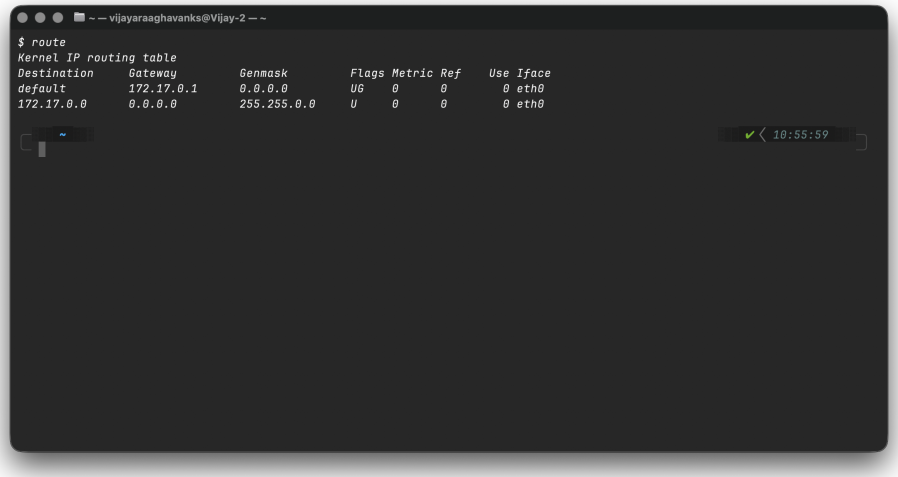


Figure 5: route – default and local subnet routes

2.6 nmap – port scanning

```
$ nmap -p 22,80,443,8080 scanme.nmap.org
```

`nmap` probes a target host's TCP ports to determine whether each is open, closed, or filtered. Against Nmap's official public test target `scanme.nmap.org`, ports 22 (SSH), 80 (HTTP) and 443 (HTTPS) were found `open`, while 8080 was `filtered` (blocked by an intermediate firewall rather than actively refused).

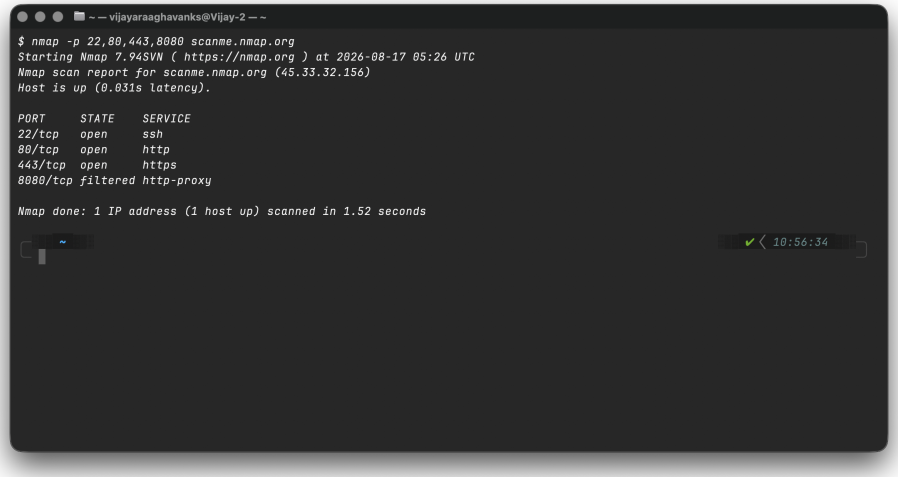


Figure 6: nmap port scan of scanme.nmap.org

2.7 tcpdump – live packet capture

```
$ tcpdump -c 5 -i eth0
```

`tcpdump` captures and prints packets crossing an interface in real time. While a background `ping` to `8.8.8.8` was running, `tcpdump` captured 5 packets on `eth0`: ICMP echo request/reply pairs and, interestingly, a DNS PTR (reverse-lookup) query/response pair for `8.8.8.8.in-addr.arpa` triggered automatically – both traffic types are visible with full source/destination and protocol detail.

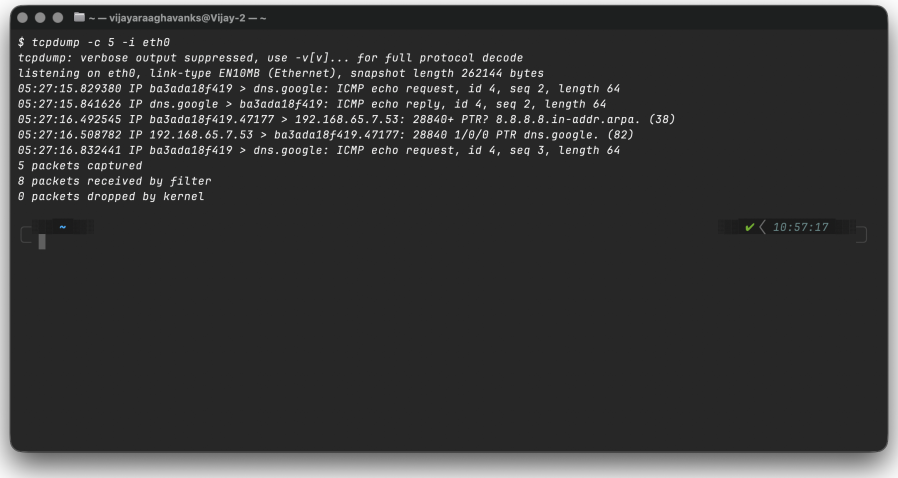


Figure 7: tcpdump – live capture of ICMP and DNS PTR traffic

2.8 dig – detailed DNS query

```
$ dig google.com
```

`dig` performs a DNS lookup and prints the full query/response structure. The `ANSWER SECTION` shows an `A` record for `google.com` resolving to `142.250.206.78` with TTL 350 s, answered by the resolver `192.168.65.7` in 2 ms over UDP.

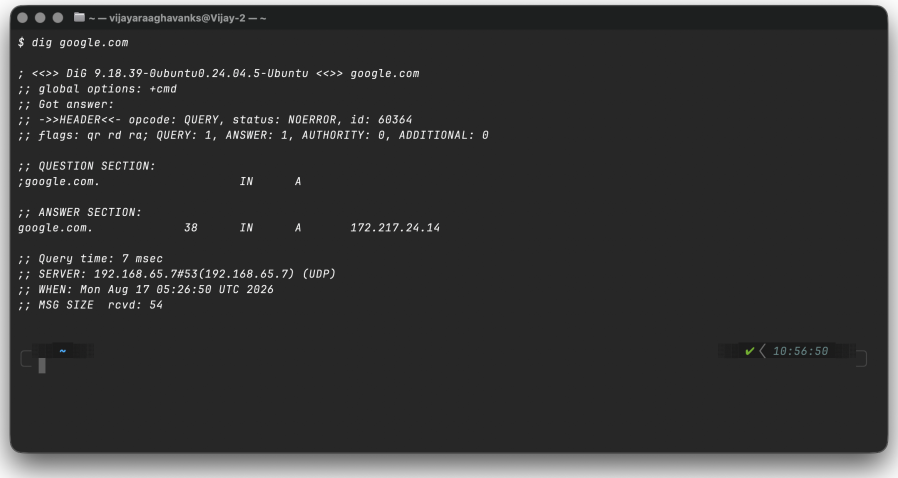


Figure 8: dig google.com – full DNS answer section

2.9 traceroute – hop-by-hop path discovery

```
$ traceroute -w 1 -q 1 -m 10 google.com
```

`traceroute` discovers the path to a destination by sending probes with increasing TTL and recording which router returns an ICMP Time-Exceeded message at each hop. Only hop 1 (the Docker gateway `172.17.0.1`) replied; hops 2–10 show `*`, since Docker Desktop's NAT layer does not generate or forward ICMP Time-Exceeded messages for the intermediate hops back into the container – the packets do reach Google (confirmed by `ping`/`dig` above) but the diagnostic ICMP replies used by traceroute are filtered.

2.10 mtr – continuous traceroute + ping

```
$ mtr -c 3 --report -n google.com
```

`mtr` combines `ping` and `traceroute`, sending repeated probes to every hop and reporting live loss/latency statistics. Unlike `traceroute`, `mtr`'s report shows only **two** hops – the gateway (`172.17.0.1`, 0% loss, avg 0.1 ms) and `google.com` itself (`142.250.206.78`, 0% loss, avg 12.8 ms) – because `mtr` silently skips non-responding intermediate hops in its summary table rather than printing `*` placeholders.

2.11 curl – HTTP request from the command line

```
$ curl -I https://www.google.com
```

`curl -I` sends an HTTP `HEAD` request and prints only the response headers. Google responded `HTTP/2 200`, and the header block reveals the content type, CSP policy, `Set-Cookie` values, and `Alt-Svc` advertisement of HTTP/3 (QUIC) support.

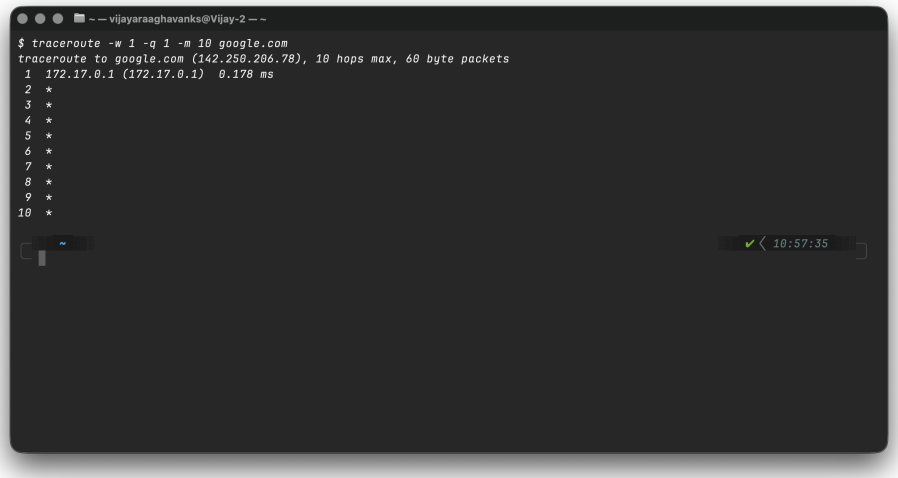


Figure 9: traceroute to google.com – only the local gateway hop responds

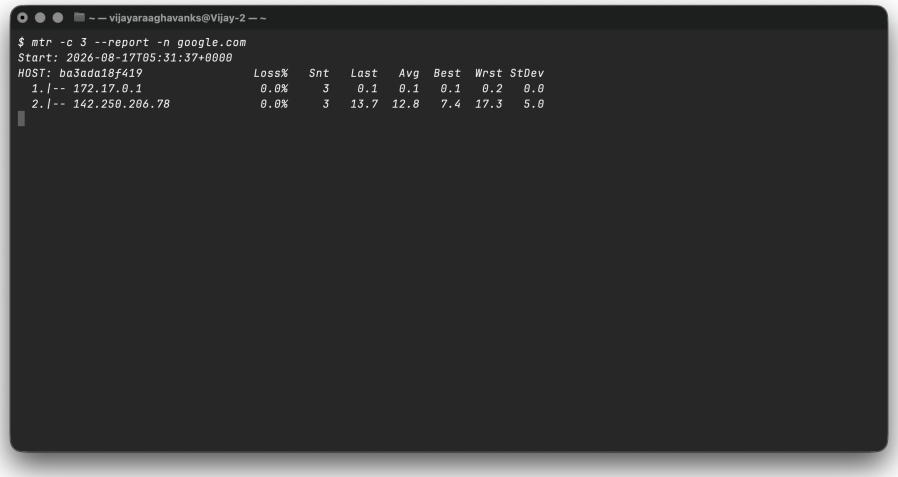


Figure 10: mtr report – gateway and destination statistics

2.12 wget – HTTP request / download client

```
$ wget -S --spider https://www.google.com
```

`wget --spider` checks whether a resource exists without downloading it, while `-S` prints the server response headers. The output shows the DNS resolution of `www.google.com` to multiple A records, a successful TLS connection on port 443, and an `HTTP/1.1 200 OK` response – confirming the resource is reachable.

2.13 tracepath – unprivileged path MTU discovery

```
$ tracepath -m 8 google.com
```

`tracepath` is a traceroute-like tool that does not require root privileges and also discovers the path MTU. As with `traceroute`, only hop 1 (`172.17.0.1`) responds within the Docker NAT environment; hops 2–8 report `no reply`, and the tool reports the default path MTU of 65535 (the container's virtual Ethernet MTU) since no smaller-MTU hop was discovered.

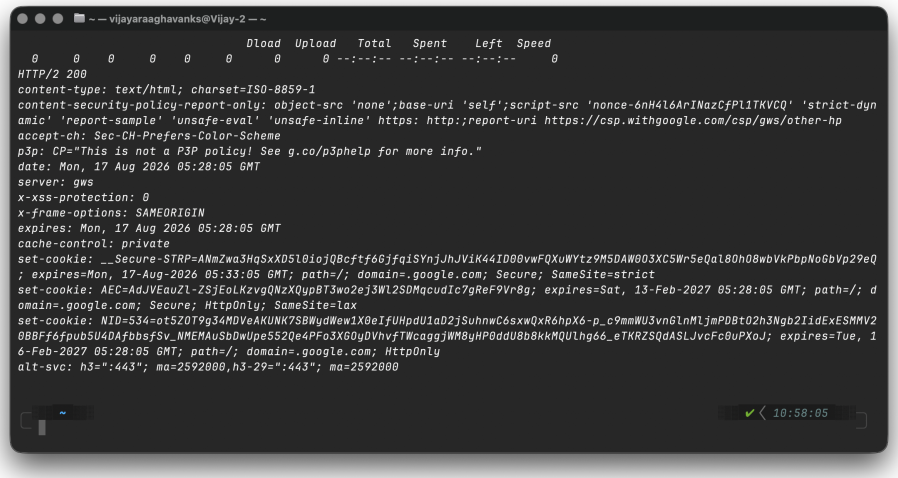


Figure 11: curl -I https://www.google.com – HTTP/2 200 response headers

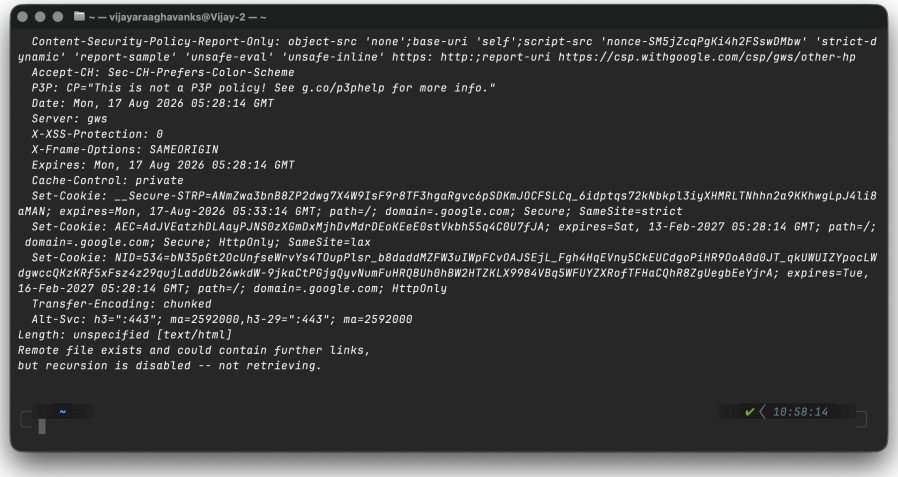


Figure 12: wget –spider -S – resolving, connecting and header dump

2.14 nslookup – interactive-style DNS lookup

```
$ nslookup google.com
```

`nslookup` queries the configured DNS server and prints a concise answer. `google.com` resolved (non-authoritatively, i.e. via a caching resolver rather than the zone's authoritative server) to `142.250.206.78` through server `192.168.65.7`.

2.15 host – simple DNS lookup utility

```
$ host google.com
```

`host` is a lightweight DNS lookup tool. A single query returned both the domain's `A` record (`142.250.206.78`) and its `MX` record (`10 smtp.google.com`), demonstrating that `host` follows all record types for a name by default rather than only `A` records.

2.16 whois – domain registration lookup

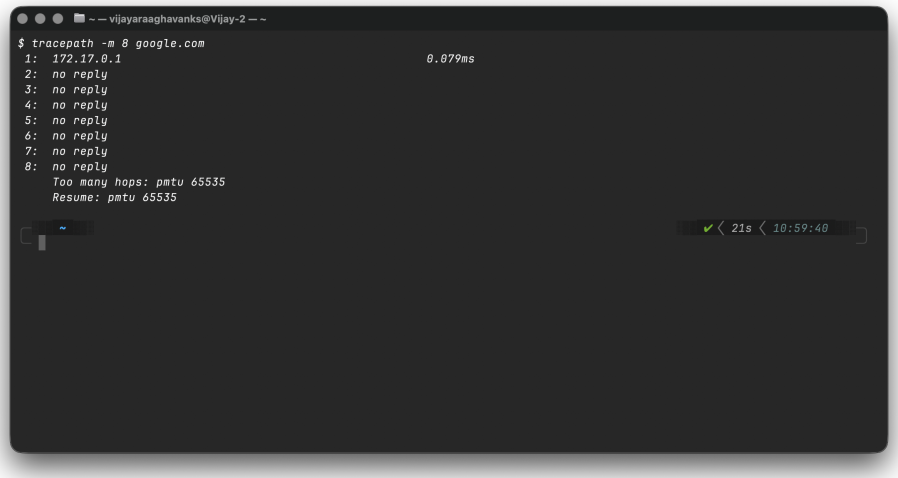


Figure 13: tracepath -m 8 google.com

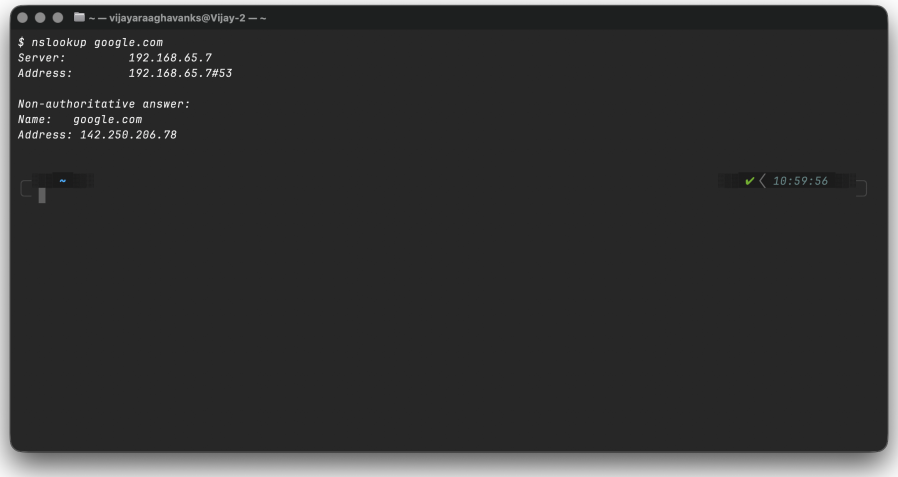


Figure 14: nslookup google.com

```
$ whois google.com
```

`whois` queries the registry/registrar's WHOIS database over TCP port 43 for domain registration details. The response lists the registrar (MarkMonitor Inc.), creation date (1997-09-15), expiry date (2028-09-14), name servers (`NS1--NS4.GOOGLE.COM`), and domain status flags such as `clientTransferProhibited`.



Figure 15: host google.com – A and MX records

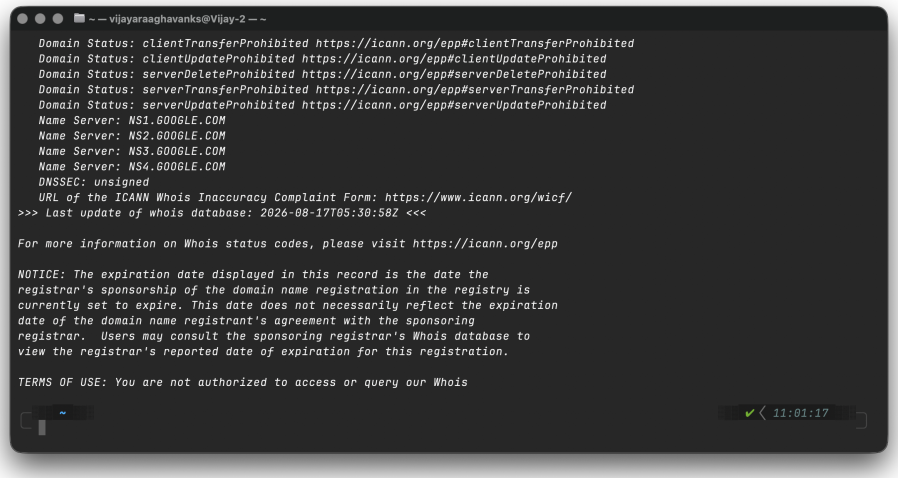


Figure 16: whois google.com – registrar and name-server details

3. Part B – Socket Programming System Calls

The following POSIX/BSD socket API functions form the building blocks of TCP/IP and UDP network programming in C. `socket()`–`bind()`–`listen()`–`accept()` set up a TCP server; `connect()` is used by a TCP client; `send()`/`recv()` (connection-oriented) and `sendto()`/`recvfrom()` (connectionless) exchange data; the remaining functions handle byte-order conversion, name resolution, and socket configuration.

Function	Purpose & Key Parameters	Return Value
`socket()`	Creates an endpoint for communication. `int socket(int domain, int type, int protocol)` – `domain`: `AF_INET`/`AF_INET6`; `type`: `SOCK_STREAM` (TCP) / `SOCK_DGRAM` (UDP).	File descriptor on success, `-1` on error.
`bind()`	Assigns a local address (IP + port) to a socket. `int bind(int sockfd, struct sockaddr *addr, socklen_t addrlen)`. Required before a server can `listen()`/`recvfrom()`.	`0` on success, `-1` on error.
`listen()`	Marks a TCP socket as passive, ready to accept incoming connections, with a backlog queue. `int listen(int sockfd, int backlog)`.	`0` on success, `-1` on error.
`connect()`	Initiates a connection (TCP) to a remote address, or fixes the peer address for a UDP socket. `int connect(int sockfd, struct sockaddr *addr, socklen_t addrlen)`.	`0` on success, `-1` on error.
`accept()`	Blocks until a connection request arrives on a listening socket; returns a new socket dedicated to that client. `int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen)`.	New connected socket fd, `-1` on error.
`close()`	Releases a socket (or any) file descriptor and its resources. `int close(int fd)`.	`0` on success, `-1` on error.
`bzero()`	Zero-fills a block of memory (commonly used to clear a `sockaddr_in` struct before populating it). `void bzero(void *s, size_t n)`.	None (void).
`htons()`	Converts a 16-bit value (e.g. port number) from Host byte order To Network byte order (big-endian). `uint16_t htons(uint16_t hostshort)`.	The value in network byte order.

Function	Purpose & Key Parameters	Return Value
`htonl()`	Converts a 32-bit value (e.g. IPv4 address) from Host to Network byte order. `uint32_t htonl(uint32_t hostlong);`	The value in network byte order.
`ntohs()`	Converts a 16-bit value from Network byte order To Host byte order – inverse of `htons()`. `uint16_t ntohs(uint16_t netshort);`	The value in host byte order.
`ntohl()`	Converts a 32-bit value from Network to Host byte order – inverse of `htonl()`. `uint32_t ntohl(uint32_t netlong);`	The value in host byte order.
`read()`	Generic POSIX call to read bytes from any file descriptor, including a connected socket. `ssize_t read(int fd, void *buf, size_t count);`	Bytes read (`0` = EOF/peer closed), `-1` on error.
`write()`	Generic POSIX call to write bytes to any file descriptor, including a connected socket. `ssize_t write(int fd, const void *buf, size_t count);`	Bytes written, `-1` on error.
`send()`	Sends data on a *connected* socket (TCP, or UDP after `connect()`), with socket-specific flags. `ssize_t send(int sockfd, const void *buf, size_t len, int flags);`	Bytes sent, `-1` on error.
`recv()`	Receives data from a connected socket. `ssize_t recv(int sockfd, void *buf, size_t len, int flags);`	Bytes received (`0` = peer closed), `-1` on error.
`sendto()`	Sends a datagram to an explicit destination address – used by connectionless UDP sockets. `ssize_t sendto(int sockfd, const void *buf, size_t len, int flags, struct sockaddr *dest, socklen_t addrlen);`	Bytes sent, `-1` on error.
`recvfrom()`	Receives a datagram and also fills in the sender's address – the UDP counterpart of `recv()`. `ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags, struct sockaddr *src, socklen_t *addrlen);`	Bytes received, `-1` on error.

Function	Purpose & Key Parameters	Return Value
`select()`	Monitors multiple file descriptors, blocking until one becomes readable/writable or a timeout expires – used for I/O multiplexing and implementing timeouts on blocking calls. `int select(int nfds, fd_set *readfds, *writefds, *exceptfds, struct timeval *timeout).`	Number of ready descriptors, `0` on timeout, `-1` on error.
`setsockopt()`	Sets a configurable option on a socket, e.g. `SO_REUSEADDR` to allow immediate rebinding of a port. `int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen).`	`0` on success, `-1` on error.
`fcntl()`	General file-descriptor control call; on sockets, commonly used to set non-blocking mode via `O_NONBLOCK`. `int fcntl(int fd, int cmd, ...).`	Depends on `cmd` (often `0`/flags on success), `-1` on error.
`getpeername()`	Retrieves the address of the peer connected to a socket. `int getpeername(int sockfd, struct sockaddr *addr, socklen_t *addrlen).`	`0` on success, `-1` on error.
`gethostname()`	Retrieves the standard host name of the local machine. `int gethostname(char *name, size_t len).`	`0` on success, `-1` on error.
`gethostbyname()`	Resolves a host name to its IP address(es) (legacy API, superseded by `getaddrinfo()` but still widely used). `struct hostent *gethostbyname(const char *name).`	Pointer to `hostent` on success, `NULL` on error.
`gethostbyaddr()`	Performs a reverse lookup, resolving an IP address back to a host name. `struct hostent *gethostbyaddr(const void *addr, socklen_t len, int type).`	Pointer to `hostent` on success, `NULL` on error.

4. Learning Outcomes

- Gained hands-on familiarity with sixteen standard Linux networking diagnostic tools and learned which layer of the network stack each one operates on – link/interface (`ifconfig`, `iwconfig`), network/routing (`route`, `netstat -r`, `traceroute`, `tracepath`, `mtr`), trans-

port/ application reachability (`ping`, `nmap`), DNS (`dig`, `nslookup`, `host`, `whois`), and raw packet capture (`tcpdump`).

- Learned to interpret `traceroute`/`tracepath`/`mtr` output in a NAT/containerised environment, and understood *why* intermediate hops can appear as `*` even though end-to-end connectivity is fully functional – an important distinction between "no route" and "ICMP diagnostic traffic filtered."
- Practised correlating live packet captures (`tcpdump`) with the higher-level tools that generated the traffic (`ping`, DNS resolution), reinforcing how ICMP and DNS operate at the packet level.
- Studied the complete socket API life cycle for both connection-oriented (TCP) and connectionless (UDP) communication, and understood why byte-order conversion functions (`htons`/`htonl`/`ntohs`/`ntohl`) are mandatory whenever multi-byte fields cross the network, given differing host endianness.
- Understood the role of auxiliary calls such as `select()` (multiplexing/timeout), `setsockopt()` (socket configuration) and `fcntl()` (non-blocking I/O) in building robust, production-quality socket applications beyond the basic `send`/`recv` loop.